

Reinforcement Learning Snake Game

Krishay Chaddha

May 2026

Contents

1	Introduction	2
1.1	Background Information	3
2	Methodology	3
2.1	Reinforcement Learning	3
2.2	State Space	3
2.3	Mathematical Formulas	4
2.3.1	Bellman Optimality Equation	4
2.3.2	Loss Function	4
2.3.3	Exploration Schedule	4
3	Dataset	5
3.0.1	rl.agent.py	5
3.0.2	q_network.py	8
3.0.3	snake_env.py	10
3.0.4	visualizer.py	13
3.1	Game Environment and Training Results	14
3.1.1	Snake Game Simulation	14
3.1.2	Training Progress Analysis	14
3.2	Performance Analysis	16
4	References	16

1 Introduction

The Snake Game is a simple game that has been around for years which makes it an interesting experiment for testing artificial intelligence and reinforcement learning. For this project, I plan to use an agent that will learn how to play the game through continuous trial and error and the agent will receive rewards for making successful decisions such as collecting food. The agent will also receive penalties for actions that lead to failure such as crashing into a wall or itself in the process. Through this experiment, I will be able to analyze how the agent improves over time. I will also use metrics such as score, reward, and survival time to measure its performance. Graphs will be generated via matplotlib and then will be used to visualize the data and determine whether the agent is successfully learning a strategy to play the game.

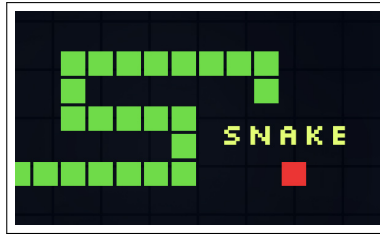


Figure 1: Snake Game Environment

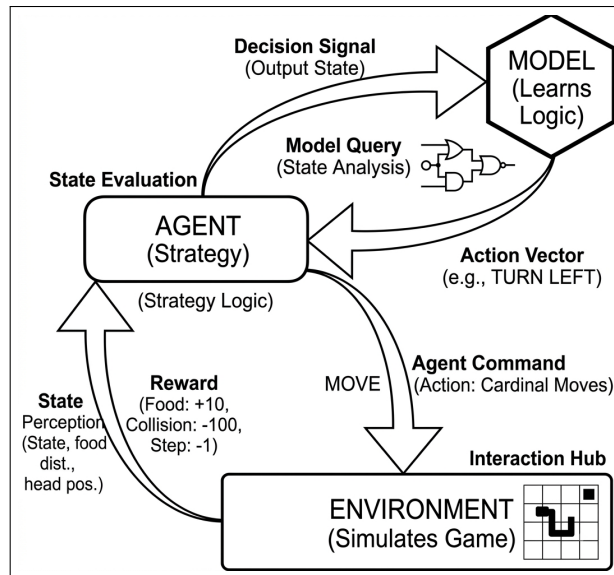


Figure 2: Reinforcement Map that will be used to train the AI based on specific moves

1.1 Background Information

The concept of reinforcement learning is based on teaching an agent how to make decisions by allowing it to interact with an environment. Instead of being given the correct action for every situation, the agent learns through consistent trial and error by receiving rewards for good actions and penalties for bad ones. With this, the agent can identify patterns and develop a strategy that improves its performance.

For this test, the snake game is well suited because every movement affects the future state of the game. For instance, collecting food increases the length of the snake which can make future movements more and more difficult. The agent therefore needs to consider both its current situation and the possible consequences of its next action. This creates a simple but effective environment for studying how an AI can learn from its own decisions.

2 Methodology

2.1 Reinforcement Learning

*In a nutshell, the reinforcement learning agent will take control of the Snake and decide which direction it should move. Every decision has a consequence: collecting food gives the agent a **positive reward**, while crashing into a wall or itself gives it a **negative reward**. By playing the game over and over again, the agent will learn from its mistakes and remember which decisions work and which ones do not. Over time, it will start choosing paths that lead to higher scores and longer survival times while avoiding paths that consistently result in negative rewards.*

2.2 State Space

The state space represents all of the information the reinforcement learning agent can use when making a decision. For the Snake Game, this includes the position of the snake, the location of the food, the direction the snake is moving, and any obstacles or dangers around it. The agent uses this information to understand the current situation and decide what action to take next. For example, if the food is to the right but there is a wall directly ahead, the agent should learn that turning right may be a better decision than continuing forward. As the agent plays more games, it will encounter different states and learn which actions produce the best rewards.

2.3 Mathematical Formulas

My agent uses three main equations to train itself and learn how to play:

2.3.1 Bellman Optimality Equation

My agent uses this equation to figure out the long-term value of every move it makes:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a') \quad (1)$$

Here s is its current situation and a is its action and r is its immediate reward. The discount factor γ helps it balance quick rewards with future survival. The final part looks ahead to find the best possible future value from its next move s' .

2.3.2 Loss Function

My agent uses this equation to measure how wrong its neural network guesses are so it can fix them:

$$\text{Loss} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (2)$$

Here y_i is its target value and $\hat{y}_i$ is what its network guessed. The variable N is the batch size of memories it looks at to make sure its training stays steady.

2.3.3 Exploration Schedule

My agent uses this equation to slowly lower how often it makes random moves as it gets smarter:

$$\epsilon = \max(5, 80 - \text{episodes}) \quad (3)$$

Here ϵ is its chance of doing something random. The floor of 5 ensures it never completely stops trying new things so it does not get stuck in bad habits.

3 Dataset

3.0.1 rl.agent.py

```
1 import torch
2 import numpy as np
3 import random
4 from collections import deque
5 from snake_env import SnakeEnvironment, Heading, GridPoint,
   GRID_SIZE
6 from q_network import SnakeQNetwork, DQNTrainer
7 from visualizer import plot_training_progress
8
9 MEMORY_CAPACITY = 100_000
10 BATCH_SIZE = 1000
11 LEARNING_RATE = 0.001
12
13 class SnakeAgent:
14     def __init__(self):
15         self.episodes = 0
16         self.epsilon = 0 # Exploration factor
17         self.gamma = 0.9 # Discount factor
18         self.memory = deque(maxlen=MEMORY_CAPACITY)
19         self.model = SnakeQNetwork(input_dim=11,
   hidden_dim=256, output_dim=3)
20         self.trainer = DQNTrainer(self.model,
   lr=LEARNING_RATE, gamma=self.gamma)
21
22     def observe_environment(self, env):
23         head = env.head
24         point_l = GridPoint(head.x - GRID_SIZE, head.y)
25         point_r = GridPoint(head.x + GRID_SIZE, head.y)
26         point_u = GridPoint(head.x, head.y - GRID_SIZE)
27         point_d = GridPoint(head.x, head.y + GRID_SIZE)
28
29         dir_l = env.heading == Heading.WEST
30         dir_r = env.heading == Heading.EAST
31         dir_u = env.heading == Heading.NORTH
32         dir_d = env.heading == Heading.SOUTH
33
34         state = [
35             # Danger ahead
36             (dir_r and env.is_collision(point_r)) or
37             (dir_l and env.is_collision(point_l)) or
38             (dir_u and env.is_collision(point_u)) or
39             (dir_d and env.is_collision(point_d)),
40
41             # Danger right
42             (dir_u and env.is_collision(point_r)) or
43             (dir_d and env.is_collision(point_l)) or
```

```

44         (dir_l and env.is_collision(point_u)) or
45         (dir_r and env.is_collision(point_d)),
46
47         # Danger left
48         (dir_d and env.is_collision(point_r)) or
49         (dir_u and env.is_collision(point_l)) or
50         (dir_r and env.is_collision(point_u)) or
51         (dir_l and env.is_collision(point_d)),
52
53         # Heading vectors
54         dir_l, dir_r, dir_u, dir_d,
55
56         # Apple relative direction
57         env.apple.x < env.head.x,
58         env.apple.x > env.head.x,
59         env.apple.y < env.head.y,
60         env.apple.y > env.head.y
61     ]
62
63     return np.array(state, dtype=int)
64
65     def remember(self, state, action, reward, next_state,
66 done):
67         self.memory.append((state, action, reward,
68 next_state, done))
69
70     def replay_experience(self):
71         sample_batch = random.sample(self.memory,
72 BATCH_SIZE) if len(self.memory) > BATCH_SIZE else
73 self.memory
74         states, actions, rewards, next_states, dones =
75 zip(*sample_batch)
76         self.trainer.train_step(states, actions, rewards,
77 next_states, dones)
78
79     def train_step(self, state, action, reward, next_state,
80 done):
81         self.trainer.train_step(state, action, reward,
82 next_state, done)
83
84     def select_action(self, state):
85         self.epsilon = max(5, 80 - self.episodes)
86         move = [0, 0, 0]
87         if random.randint(0, 200) < self.epsilon:
88             move_idx = random.randint(0, 2)
89         else:
90             state_tensor = torch.tensor(state,
91 dtype=torch.float)
92             prediction = self.model(state_tensor)
93             move_idx = torch.argmax(prediction).item()

```

```

85         move[move_idx] = 1
86         return move
87
88
89 def train():
90     scores = []
91     average_scores = []
92     total_score = 0
93     best_score = 0
94     agent = SnakeAgent()
95     env = SnakeEnvironment()
96
97     while True:
98         old_state = agent.observe_environment(env)
99         action = agent.select_action(old_state)
100         reward, done, score = env.step(action)
101         new_state = agent.observe_environment(env)
102
103         agent.train_step(old_state, action, reward,
104 new_state, done)
105         agent.remember(old_state, action, reward,
106 new_state, done)
107
108         if done:
109             env.reset()
110             agent.episodes += 1
111             agent.replay_experience()
112
113             if score > best_score:
114                 best_score = score
115                 agent.model.save_checkpoint()
116
117             scores.append(score)
118             total_score += score
119             avg_score = total_score / agent.episodes
120             average_scores.append(avg_score)
121
122             print(f'Episode: {agent.episodes} | Score:
123 {score} | Best: {best_score} | Avg: {avg_score:.2f}')
124             plot_training_progress(scores, average_scores)
125
126 if __name__ == '__main__':
127     train()

```

3.0.2 q_network.py

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 import os
6 import numpy as np
7
8 class SnakeQNetwork(nn.Module):
9     def __init__(self, input_dim=11, hidden_dim=256,
10         output_dim=3):
11         super().__init__()
12         self.fc1 = nn.Linear(input_dim, hidden_dim)
13         self.fc2 = nn.Linear(hidden_dim, output_dim)
14
15     def forward(self, x):
16         x = F.relu(self.fc1(x))
17         return self.fc2(x)
18
19     def save_checkpoint(self,
20         filepath='./saved_models/snake_dqn.pth'):
21         os.makedirs(os.path.dirname(filepath),
22             exist_ok=True)
23         torch.save(self.state_dict(), filepath)
24
25 class DQNTrainer:
26     def __init__(self, network, lr=0.001, gamma=0.9):
27         self.network = network
28         self.gamma = gamma
29         self.optimizer = optim.Adam(network.parameters(),
30             lr=lr)
31         self.loss_fn = nn.MSELoss()
32
33     def train_step(self, states, actions, rewards,
34         next_states, dones):
35         states = torch.tensor(np.array(states),
36             dtype=torch.float)
37         next_states = torch.tensor(np.array(next_states),
38             dtype=torch.float)
39         actions = torch.tensor(np.array(actions),
40             dtype=torch.long)
41         rewards = torch.tensor(np.array(rewards),
42             dtype=torch.float)
43
44         if len(states.shape) == 1:
45             states = states.unsqueeze(0)
46             next_states = next_states.unsqueeze(0)
47             actions = actions.unsqueeze(0)
48             rewards = rewards.unsqueeze(0)
```



```

40         done = (done,)
41
42         predictions = self.network(states)
43         targets = predictions.clone()
44
45         for i in range(len(done)):
46             q_value = rewards[i]
47             if not done[i]:
48                 q_value = rewards[i] + self.gamma *
49                 torch.max(self.network(next_states[i]))
50                 targets[i][torch.argmax(actions[i]).item()] =
51                 q_value
52
53         self.optimizer.zero_grad()
54         loss = self.loss_fn(targets, predictions)
55         loss.backward()
56         self.optimizer.step()

```

3.0.3 snake_env.py

```
1 import pygame
2 import random
3 from enum import Enum
4 from collections import namedtuple
5 import numpy as np
6
7 pygame.init()
8 UI_FONT = pygame.font.SysFont('arial', 20)
9
10 class Heading(Enum):
11     EAST = 1
12     WEST = 2
13     NORTH = 3
14     SOUTH = 4
15
16 GridPoint = namedtuple('GridPoint', 'x, y')
17
18 COLOR_BG = (15, 15, 20)
19 COLOR_SNAKE_HEAD = (0, 230, 150)
20 COLOR_SNAKE_BODY = (0, 180, 120)
21 COLOR_APPLE = (230, 50, 50)
22 COLOR_TEXT = (240, 240, 240)
23
24 GRID_SIZE = 20
25 GAME_SPEED = 40
26
27 class SnakeEnvironment:
28     def __init__(self, width=640, height=480):
29         self.width = width
30         self.height = height
31         self.screen = pygame.display.set_mode((self.width,
32 self.height))
33         pygame.display.set_caption('Autonomous Snake -
34 Reinforcement Learning')
35         self.clock = pygame.time.Clock()
36         self.reset()
37
38     def reset(self):
39         self.heading = Heading.EAST
40         self.head = GridPoint(self.width // 2, self.height
41 // 2)
42         self.body = [
43             self.head,
44             GridPoint(self.head.x - GRID_SIZE, self.head.y),
45             GridPoint(self.head.x - (2 * GRID_SIZE),
46 self.head.y)
47         ]
48         self.score = 0
```

```

45         self.apple = None
46         self._spawn_apple()
47         self.steps_count = 0 # Steps taken without eating
food
48
49     def _spawn_apple(self):
50         x = random.randint(0, (self.width - GRID_SIZE) //
GRID_SIZE) * GRID_SIZE
51         y = random.randint(0, (self.height - GRID_SIZE) //
GRID_SIZE) * GRID_SIZE
52         self.apple = GridPoint(x, y)
53         if self.apple in self.body:
54             self._spawn_apple()
55
56     def step(self, action):
57         self.steps_count += 1
58         for event in pygame.event.get():
59             if event.type == pygame.QUIT:
60                 pygame.quit()
61                 quit()
62
63         self._update_heading(action)
64         self.body.insert(0, self.head)
65
66         reward = 0
67         is_done = False
68
69         # Collision with wall/self OR stalled without
eating food for too long
70         if self.is_collision() or self.steps_count > 100 *
len(self.body):
71             is_done = True
72             reward = -10
73             return reward, is_done, self.score
74
75         # Check if apple is eaten
76         if self.head == self.apple:
77             self.score += 1
78             reward = 10
79             self.steps_count = 0 # FIX: Reset step timer
when food is eaten
80             self._spawn_apple()
81         else:
82             self.body.pop()
83
84         self._render()
85         self.clock.tick(GAME_SPEED)
86         return reward, is_done, self.score
87
88     def is_collision(self, point=None):

```

```

89         if point is None:
90             point = self.head
91             if point.x >= self.width or point.x < 0 or point.y
92             >= self.height or point.y < 0:
93                 return True
94             if point in self.body[1:]:
95                 return True
96             return False
97
98     def _render(self):
99         self.screen.fill(COLOR_BG)
100         for segment in self.body:
101             color = COLOR_SNAKE_HEAD if segment ==
102             self.head else COLOR_SNAKE_BODY
103             pygame.draw.rect(self.screen, color,
104                             pygame.Rect(segment.x, segment.y, GRID_SIZE, GRID_SIZE))
105
106             pygame.draw.rect(self.screen, COLOR_APPLE,
107                             pygame.Rect(self.apple.x, self.apple.y, GRID_SIZE,
108                             GRID_SIZE))
109
110             score_surface = UI_FONT.render(f'Score:
111             {self.score}', True, COLOR_TEXT)
112             self.screen.blit(score_surface, (10, 10))
113             pygame.display.flip()
114
115     def _update_heading(self, action):
116         order = [Heading.EAST, Heading.SOUTH, Heading.WEST,
117         Heading.NORTH]
118         idx = order.index(self.heading)
119
120         if np.array_equal(action, [1, 0, 0]):
121             new_heading = order[idx]
122         elif np.array_equal(action, [0, 1, 0]):
123             new_heading = order[(idx + 1) % 4]
124         else:
125             new_heading = order[(idx - 1) % 4]
126
127         self.heading = new_heading
128
129         x, y = self.head.x, self.head.y
130         if self.heading == Heading.EAST:
131             x += GRID_SIZE
132         elif self.heading == Heading.WEST:
133             x -= GRID_SIZE
134         elif self.heading == Heading.SOUTH:
135             y += GRID_SIZE
136         elif self.heading == Heading.NORTH:
137             y -= GRID_SIZE
138         self.head = GridPoint(x, y)

```

3.0.4 visualizer.py

```
1 import matplotlib.pyplot as plt
2
3 def plot_training_progress(scores, average_scores):
4     plt.clf()
5     plt.title('Training Progress - Deep Q-Learning')
6     plt.xlabel('Episodes')
7     plt.ylabel('Score')
8     plt.plot(scores, label='Score per Episode', alpha=0.5,
9             color='cornflowerblue')
10    plt.plot(average_scores, label='Cumulative Average',
11           color='firebrick', linewidth=2)
12    plt.ylim(bottom=0)
13    plt.text(len(scores) - 1, scores[-1], str(scores[-1]))
14    plt.text(len(average_scores) - 1, average_scores[-1],
15           f'{average_scores[-1]:.1f}')
16    plt.legend(loc='upper left')
17    plt.draw()
18    plt.pause(0.1)
```

3.1 Game Environment and Training Results

3.1.1 Snake Game Simulation

Figure 3 shows the graphical user interface for my reinforcement learning project implemented via PyGame. The window displays the custom environment where the autonomous snake navigates the grid to collect red apple targets while avoiding collisions with the walls and its own body.

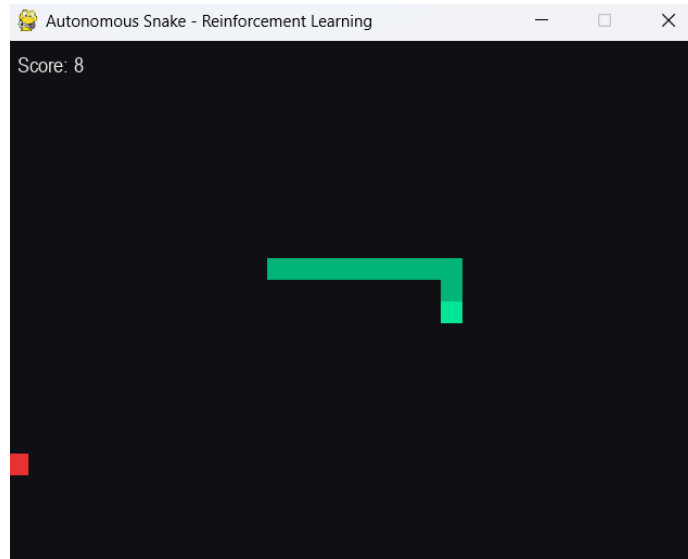


Figure 3: The autonomous snake game interface during execution showing the agent tracking a score of 8.

3.1.2 Training Progress Analysis

Figures 4 and 5 illustrates the quantitative progression of my deep Q-learning model across multiple training episodes. The graph tracks both the episodic score fluctuations in light blue and the steady upward trend of the cumulative moving average in dark red. As the training episodes accumulate past 100, the agent successfully overcomes initial random policies and significantly improves its performance.

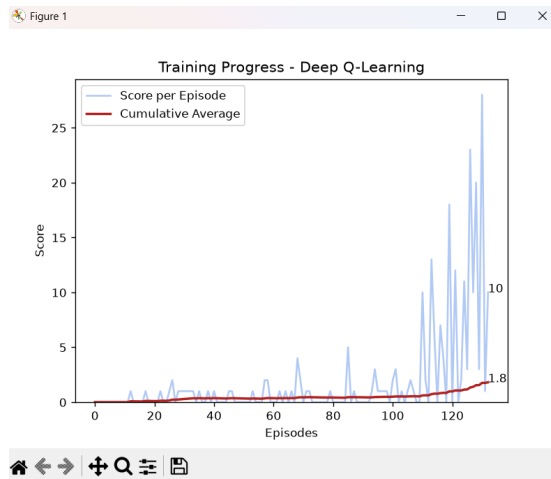


Figure 4: Training progress plot showing episodic scores and cumulative averages over time.

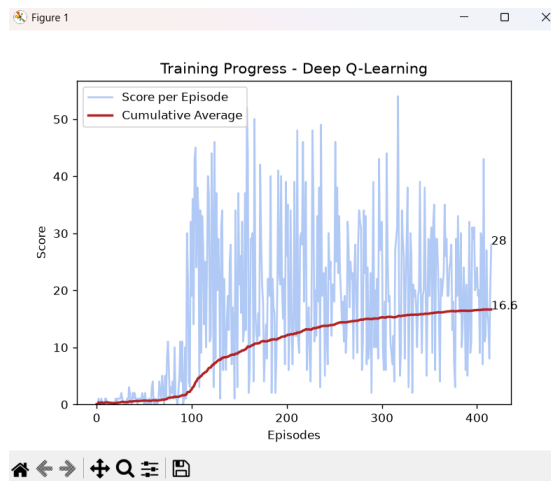


Figure 5: Longer Duration Run

This graph shows the rapid increase of the number of points the Snake receives over test results and episodes. It also shows that the number of episodes is positively correlated with the score. With an increase in episodes, the snake receives more data.

3.2 Performance Analysis

For the performance, the main factors were used to evaluate how the reinforcement learning agent learned over time and the amount of time the snake was able to survive without crashing into itself or the surrounding walls. These factors helped in determining whether the agent was improving its decision making and learning a successful strategy through repeated training.

4 References

- Mnih, Volodymyr, et al. “Human-Level Control through Deep Reinforcement Learning.” *Nature*, vol. 518, no. 7540, 2015, pp. 529–533. <https://doi.org/10.1038/nature14236>.
- Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed., MIT Press, 2018.
- Watkins, Christopher J. C. H., and Peter Dayan. “Q-Learning.” *Machine Learning*, vol. 8, 1992, pp. 279–292. <https://doi.org/10.1007/BF00992698>.
- PyTorch. “PyTorch Documentation.” PyTorch, <https://docs.pytorch.org/docs/stable/>. Accessed 13 Aug. 2026.
- Pygame. “Pygame Documentation.” Pygame, <https://www.pygame.org/docs/>. Accessed 13 Aug. 2026.
- Matplotlib Development Team. “Matplotlib Documentation.” Matplotlib, <https://matplotlib.org/stable/>. Accessed 13 Aug. 2026.